PADCOM V22 - GUIA DE IMPLEMENTAÇÃO PARA REPLIT AGENT

Versão: 1.0

Data: 11 de Abril de 2026

Status: Pronto para Implementação

Destinatário: Replit Agent + Equipe Técnica

VISÃO GERAL DO SISTEMA

O PADCOM V22 é um sistema completo de **check-in e acompanhamento longitudinal** que transforma o fluxo de anamnese de um formulário respondido do zero para um **banco de dados clínico estruturado com validação por check-in**.

Fluxo Principal

1. **ENTRADA (Check-in):** Paciente marca as doenças que tem (não responde perguntas abertas)
2. **ENGINE (Motor):** Sistema gera cards mensais para Admin, Enfermeira, Consultor
3. **OUTPUT (Dashboard):** Médico vê timeline completa, evolução de exames, histórico de mudanças

Diferencial

- **Sem redundância:** Paciente responde uma vez, valida mudanças depois
- **Rastreabilidade:** Log de quem perguntou, quando e o que respondeu
- **Correlação clínica:** Você (médico) vê tudo integrado para correlacionar com o paciente
- **Acompanhamento automático:** Cards gerados mensalmente com perguntas inteligentes

---

ARQUITETURA TÉCNICA

Stack

- **Backend:** Express 5 + PostgreSQL + Drizzle ORM
- **Frontend:** React 19 + Tailwind CSS 4 + shadcn/ui

- **Banco:** 14 tabelas novas (master data + check-in + acompanhamento)
- **API:** 15+ rotas RESTful

Estrutura de Dados

Camada	Tabelas	Função
Master Data	`patologias_master`, `cirurgias_master`, `canceres_master`, `sintomas_master`	Banco de referência de todas as doenças, cirurgias, cânceres e sintomas
Check-in	`paciente_patologias_checkin`, `paciente_cirurgias_checkin`, `paciente_canceres_checkin`, `paciente_sintomas_tracking`	Dados que o paciente marca/registra
Acompanhamento	`acompanhamento_cards_mensais`, `historico_mudancas_patologias`, `historico_mudancas_sintomas`, `acompanhamento_timeline`	Cards, histórico de mudanças e timeline
Regras	`regras_motor_patologias`, `perguntas_cards_template`	Configurações do motor clínico e templates de perguntas

---

FASE 1: PREPARAÇÃO DO BANCO DE DADOS

1.1 Executar Migrations

Copie o arquivo `migrations_v22.sql` para seu ambiente e execute:

Bash

```bash
psql -U seu_usuario -d seu_banco -f migrations_v22.sql
```

O que será criado:

- 14 tabelas novas com índices otimizados
- Dados iniciais (patologias, cirurgias, cânceres, sintomas, templates de perguntas)

- Constraints e validações

1.2 Verificar Criação

```SQL
SELECT table_name FROM information_schema.tables
WHERE table_schema = 'public' AND table_name LIKE '%v22%' OR table_name LIKE '%
```

Você deve ver 14 tabelas.

FASE 2: INTEGRAÇÃO COM DRIZZLE ORM

2.1 Adicionar Schemas

Copie o arquivo `drizzle_schemas_v22.ts` para:

```Plain Text
lib/db/src/schema/padcom_v22.ts
```

2.2 Exportar no Index

Edite `lib/db/src/index.ts` e adicione:

```TypeScript
export {
  patologiasMasterTable,
  cirurgiasMasterTable,
  canceresMasterTable,
  sintomasMasterTable,
  pacientePatologiasCheckinTable,
  pacienteCirugiasCheckinTable,
  pacienteCanceresCheckinTable,
  pacienteSintomasTrackingTable,
  acompanhamentoCardsMensaisTable,
  historicoMudancasPatologiasTable,
  historicoMudancasSintomasTable,
  acompanhamentoTimelineTable,
  regrasMotorPatologiasTable,
  perguntasCardsTemplateTable,
  // ... relations
  patologiasRelations,
```

```
  cirurgiasRelations,
  // ... etc
} from "./schema/padcom_v22";
```

2.3 Testar Conexão

TypeScript

```
import { db, patologiasMasterTable } from "@workspace/db";

const patologias = await db.select().from(patologiasMasterTable).limit(5);
console.log(patologias); // Deve retornar as patologias iniciais
```

FASE 3: IMPLEMENTAR ROTAS API

3.1 Adicionar Rotas

Copie o arquivo `routes_v22_checkin.ts` para:

Plain Text

```
artifacts/api-server/src/routes/v22_checkin.ts
```

3.2 Registrar no Express

Edite `artifacts/api-server/src/index.ts` e adicione:

TypeScript

```
import v22Router from "./routes/v22_checkin";

app.use("/api", v22Router);
```

3.3 Testar Rotas

Bash

```
curl http://localhost:3000/api/v22/patologias-master
```

Você deve receber um JSON com todas as patologias.

FASE 4: CRIAR COMPONENTES REACT

4.1 Adicionar Componentes

Copie o arquivo `components_v22_ui.tsx` para:

Plain Text

```
client/src/components/v22/CheckinPatologias.tsx
client/src/components/v22/CardsMensais.tsx
client/src/components/v22/TimelineAcompanhamento.tsx
client/src/components/v22/DashboardMedico.tsx
```

4.2 Criar Página de Acompanhamento

Crie `client/src/pages/Acompanhamento.tsx` :

TypeScript

```
import { CheckinPatologias } from "@/components/v22/CheckinPatologias";
import { CardsMensais } from "@/components/v22/CardsMensais";
import { TimelineAcompanhamento } from "@/components/v22/TimelineAcompanhamento
import { DashboardMedico } from "@/components/v22/DashboardMedico";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@/components/ui/tabs"

export default function Acompanhamento( ) {
  const pacienteId = 1; // Obter do contexto/URL

  return (
    <Tabs defaultValue="checkin" className="w-full">
      <TabsList>
        <TabsTrigger value="checkin">Check-in</TabsTrigger>
        <TabsTrigger value="cards">Cards</TabsTrigger>
        <TabsTrigger value="timeline">Timeline</TabsTrigger>
        <TabsTrigger value="dashboard">Dashboard</TabsTrigger>
      </TabsList>

      <TabsContent value="checkin">
        <CheckinPatologias pacienteId={pacienteId} />
      </TabsContent>

      <TabsContent value="cards">
        <CardsMensais pacienteId={pacienteId} />
      </TabsContent>

      <TabsContent value="timeline">
```

```
        <TimelineAcompanhamento pacienteId={pacienteId} />
      </TabsContent>

      <TabsContent value="dashboard">
        <DashboardMedico pacienteId={pacienteId} />
      </TabsContent>
    </Tabs>
  );
}
```

4.3 Registrar Rota

Edite `client/src/App.tsx` :

TypeScript

```
import Acompanhamento from "@/pages/Acompanhamento";

<Route path="/acompanhamento" component={Acompanhamento} />
```

FASE 5: JOB DE GERAÇÃO DE CARDS MENSAIS

5.1 Criar Script

Crie `artifacts/api-server/src/jobs/gerar_cards_mensais.ts` :

TypeScript

```
import { db, pacientesTable, acompanhamentoCardsMensaisTable, perguntasCardsTem
import { eq } from "drizzle-orm";

export async function gerarCardsMensais() {
  console.log("[JOB] Iniciando geração de cards mensais...");

  try {
    // 1. Buscar todos os pacientes ativos
    const pacientes = await db
      .select({ id: pacientesTable.id })
      .from(pacientesTable)
      .where(eq(pacientesTable.ativo, true));

    const mesAno = new Date();
    mesAno.setDate(1); // Primeiro dia do mês

    // 2. Para cada paciente, gerar cards
```

```
    for (const paciente of pacientes) {
      // Verificar se já existe card para este mês
      const existente = await db
        .select()
        .from(acompanhamentoCardsMensaisTable)
        .where(
          and(
            eq(acompanhamentoCardsMensaisTable.pacienteId, paciente.id),
            eq(acompanhamentoCardsMensaisTable.mesAno, mesAno)
          )
        );

      if (existente.length === 0) {
        // Buscar templates de perguntas
        const perguntasAdmin = await db
          .select()
          .from(perguntasCardsTemplateTable)
          .where(
            and(
              eq(perguntasCardsTemplateTable.tipoCard, "ADMIN"),
              eq(perguntasCardsTemplateTable.ativo, true)
            )
          );

        // Criar cards (ADMIN, ENFERMEIRA, CONSULTOR)
        await db.insert(acompanhamentoCardsMensaisTable).values([
          {
            pacienteId: paciente.id,
            mesAno,
            tipoCard: "ADMIN",
            perguntasSugeridas: perguntasAdmin,
            status: "pendente",
            criadoEm: new Date(),
          },
          // ... ENFERMEIRA, CONSULTOR
        ]);

        console.log(`✓ Cards gerados para paciente ${paciente.id}`);
      }
    }

    console.log("[JOB] Geração concluída");
  } catch (error) {
    console.error("[JOB] Erro:", error);
  }
}
```

5.2 Agendar com Cron

Edite `artifacts/api-server/src/index.ts` :

TypeScript

```
import cron from "node-cron";
import { gerarCardsMensais } from "./jobs/gerar_cards_mensais";

// Executar 1º dia de cada mês às 00:00
cron.schedule("0 0 1 * *", () => {
  gerarCardsMensais();
});
```

FASE 6: INTEGRAÇÃO COM MOTOR CLÍNICO

6.1 Atualizar Rota de Anamnese

Edite `artifacts/api-server/src/routes/anamnese.ts` e adicione ao final:

TypeScript

```
// Ao criar anamnese, processar check-in no motor
router.post("/anamnese/criar", async (req, res) => {
  // ... código existente ...

  // Após criar anamnese, processar no motor V22
  if (anamneseId) {
    await fetch("http://localhost:3000/api/v22/motor-clinico/processar-checkin"
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ pacienteId, anamneseId } ),
    });
  }

  res.json({ sucesso: true, anamneseId });
});
```

FASE 7: TESTES E VALIDAÇÃO

7.1 Teste de Check-in

Bash

```bash
curl -X POST http://localhost:3000/api/v22/checkin/patologias \
  -H "Content-Type: application/json" \
  -d '{
    "pacienteId": 1,
    "patologias": [
      { "patologiaId": 1, "intensidade": 3, "observacoes": "Controlada" },
      { "patologiaId": 5, "intensidade": 2, "observacoes": "" }
    ]
  }'
```

Resposta esperada:

```JSON
{
  "sucesso": true,
  "mensagem": "2 patologias registradas",
  "dados": [...]
}
```

7.2 Teste de Dashboard

```Bash
curl http://localhost:3000/api/v22/dashboard/paciente/1
```

Resposta esperada:

```JSON
{
  "pacienteId": 1,
  "patologiasAtuais": [...],
  "ultimosSintomas": [...],
  "cardsPendentes": 3,
  "timelineRecente": [...]
}
```

7.3 Teste de Frontend

1. Acesse `http://localhost:3000/acompanhamento`
2. Clique em "Check-in"

3. Selecione algumas patologias
4. Clique em "Salvar"
5. Verifique se os dados aparecem em "Dashboard"

CHECKLIST DE IMPLEMENTAÇÃO

- [ ] Executar migrations SQL
- [ ] Adicionar schemas Drizzle ORM
- [ ] Registrar schemas no index.ts
- [ ] Copiar rotas TypeScript
- [ ] Registrar rotas no Express
- [ ] Copiar componentes React
- [ ] Criar página de Acompanhamento
- [ ] Registrar rota no App.tsx
- [ ] Criar job de geração de cards
- [ ] Agendar cron job
- [ ] Integrar com motor clínico
- [ ] Testar check-in (API)
- [ ] Testar dashboard (API)
- [ ] Testar frontend (UI)
- [ ] Validar com paciente piloto

DADOS INICIAIS A POPULAR

Após executar as migrations, você terá:

Patologias Master (14 exemplos)

- CARDIACAS: HAS, Arritmia, Angina
- METABOLICAS: Diabetes T1/T2, Resistência Insulínica
- ENDOCRINAS: Hipotireoidismo, Hipertireoidismo
- AUTOIMUNES: Lúpus, Artrite Reumatoide

- ONCOLOGICAS: Câncer de Mama, Próstata, Pulmão
- ORTOPEDICAS: Artrose, Osteoporose

Sintomas Master (10 exemplos)

- ENERGIA: Fadiga
- SONO: Insônia
- PSICOLOGIA: Ansiedade, Depressão
- DOR: Dor Muscular, Cefaleia
- METABOLISMO: Ganho de Peso, Perda de Peso
- DIGESTIVO: Constipação, Diarreia

Perguntas de Cards (11 templates)

- **ADMIN:** Pressão arterial, Peso, Medicações, Emergências
- **ENFERMEIRA:** Energia, Sono, Sintomas novos, Atividade física
- **CONSULTOR:** Motivação, Dúvidas, Dificuldades

Para adicionar mais dados: Edite `migrations_v22.sql` na seção "SEED DATA" antes de executar.

PRÓXIMOS PASSOS APÓS IMPLEMENTAÇÃO

1. **Testar com paciente piloto** - Validar fluxo completo
2. **Ajustar templates de perguntas** - Conforme feedback
3. **Adicionar mais patologias** - Importar da V22 completa
4. **Integrar com exames** - Ligar cards com resultados de exames
5. **Criar relatórios** - Gráficos de evolução por sintoma/patologia
6. **Implementar notificações** - Alertar quando cards estão pendentes

---

SUPORTE E DÚVIDAS

Se tiver dúvidas durante a implementação:

1. Verifique os logs do servidor: `tail -f .manus-logs/devserver.log`
2. Verifique os logs do banco: `psql -U seu_usuario -d seu_banco -c "SELECT * FROM pg_stat_statements LIMIT 10;"`

3. Teste as rotas com curl antes de testar no frontend
4. Valide os dados no banco com queries SQL diretas

FIM DO GUIA

Você está pronto para implementar o PADCOM V22. Boa sorte! 🚀